

ViDAL: A Visual Dynamics-Grounded Action Latent Space for Vision-Language-Action Models

Yuan Xu^{1,2*} Yixiang Chen^{1,2} Qisen Ma^{1,2} Jiabing Yang^{1,2} Peiyan Li^{1,2}
 Kai Wang^{1,2} Jianhua Yang^{1,2} Jianlou Si³ Jun Huang³
 Jing Liu^{1,2,4} Nianfeng Liu^{1,2,4} Yan Huang^{1,2,4} Liang Wang^{1,2}

¹CASIA ²UCAS ³Alibaba Group ⁴FiveAges

Abstract

Vision-Language-Action (VLA) models have become a central paradigm for robot policy learning, which predict actions in three forms: raw action chunks, discrete action tokens, or continuous action latents. However, existing action representations primarily model action trajectories, with limited consideration of the visual dynamics induced by these actions. We introduce ViDAL, a Visual Dynamics-grounded Action Latent Space that anchors continuous action latents in the future visual dynamics of the scene. Specifically, ViDAL learns action latent space by training an Action Variational Autoencoder (Action VAE) to reconstruct action chunks while aligning its latent with future scene dynamics. When integrated into downstream robot policies, the proposed Action VAE serves as a plug-in action interface compatible with multiple VLA architectures and enables optional future-video prediction as an additional capability. Empirically, ViDAL outperforms competitive baselines on LIBERO with 98.1% average success, improves a multi-task $\pi_{0.5}$ policy on RoboTwin 2.0 from 54.3% to 65.5% (Clean) and from 33.2% to 43.1% (Random) success rates over 50 dual-arm tasks, and yields 20.0% and 23.4% absolute success-rate gains on real-world single-arm Franka and dual-arm ARX robot platforms.

1 Introduction

Vision-Language-Action (VLA) models [1–6] have emerged as a popular paradigm for generalist robot manipulation, directly predicting actions from visual observations and language instructions. Since actions define the interface between the VLA policy and robot control, their representation determines both the learning target and the executable control output. In recent VLA systems, action representations usually take three predominant forms: raw action chunks regressed by a continuous head [4, 5, 7], discrete action tokens produced by a tokenizer [8–11], or continuous action latents produced by a learned autoencoder [12]. These representations primarily model action trajectories, but manipulation actions are meaningful through their effects on the scene, such as object motion, contact changes, and task-state transitions. Accordingly, as VLA gradually scales to long-horizon, multi-task and multi-scenario manipulation [13–16], an effective action representation should account for not only the action trajectory but also the visual dynamics induced by the action.

Recent works have pursued two complementary directions: learning compact action representations and incorporating visual dynamics into VLA policies. As shown in Figure 1(a), early action tokenizers or autoencoders compress action chunks into compact tokens or latents [8–12], yielding representations primarily optimized for action reconstruction, without modeling visual dynamics. As shown in Figure 1(b), VLAs with visual prediction use future video or world-model signals to

*yuan.xu@nlpr.ia.ac.cn

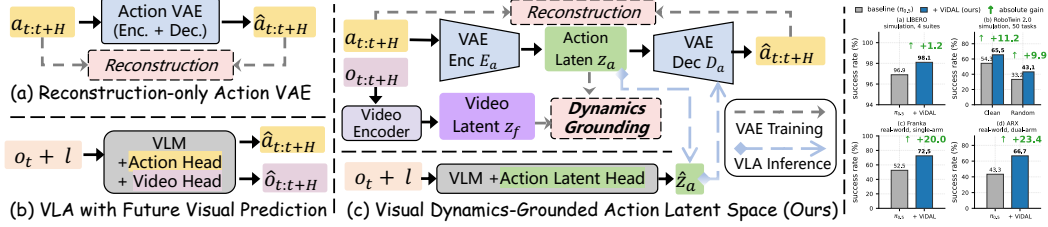


Figure 1: **Overview of ViDAL.** (a) Action VAE supervised only by reconstruction. (b) VLAs supervised with visual prediction signals. (c) ViDAL (ours) grounds the action latent space in future visual dynamics, and uses the Action VAE as a plug-in action representation at deployment. Right column: consistent success-rate gains on simulation benchmarks and realrobot platforms.

guide policy learning [17–22]. Together, these directions suggest that visual dynamics benefit action learning, but leave open how they should shape the action representation. Since actions are visually contextual, the challenge is learning an executable, visual dynamics-grounded action latent space.

We propose ViDAL (Visual Dynamics-grounded Action Latent Space, Figure 1(c)) to inject visual dynamics directly into the action latent during representation learning. In ViDAL, a frozen video VAE extracts the initial and future visual latents from each robot trajectory, and an Action VAE encodes the corresponding action chunk into a continuous action latent. A lightweight visual-dynamics predictor then maps the initial visual latent and the action latent to the future visual latent, training the action latent to preserve executable actions while explaining future scene dynamics conditioned on the current observation. For downstream policy learning, only the frozen Action VAE is required: its encoder produces action latent targets for VLA training, and its decoder maps VLA-predicted latents to executable robot actions. Thus, the decoupled design lets ViDAL plug into both regression (e.g., OpenVLA-OFT [23]) and flow-matching (e.g., $\pi_{0.5}$ [5]) action experts without modifying the VLM backbone. The video VAE also enables future-video prediction for visualizing the learned dynamics.

We evaluate ViDAL on two simulation benchmarks (LIBERO [24], RoboTwin 2.0 [25]) and two real-world platforms (single-arm Franka, dual-arm ARX), using both regression (OpenVLA-OFT [23], StarVLA-OFT [26]) and flow-matching ($\pi_{0.5}$ [5]) action experts as the downstream policy. Across all four settings, ViDAL consistently improves the underlying VLA policies and surpasses competitive action-representation baselines on LIBERO (Figure 1, right): on the higher-dimensional dual-arm RoboTwin 2.0, it lifts a multi-task $\pi_{0.5}$ policy by 11.2% Clean and 9.9% Random over 50 tasks, and on real robots it delivers 20.0% absolute gain on Franka and 23.4% on ARX. These significant gains across simulation and real, single- and dual-arm settings establish visual-dynamics grounding as a better design principle for action representations.

Our contributions are summarized as follows:

- We introduce ViDAL, which, to our knowledge, is the first approach to ground action latents in future visual dynamics for VLA models, learned via an Action VAE jointly trained on action reconstruction and future visual latent prediction.
- We propose a plug-in action-latent interface for downstream VLA policies, where the frozen Action VAE serves as the action representation for both regression and flow-matching action experts, requiring no changes to the VLM backbone.
- We conduct extensive experiments in both simulation (LIBERO, RoboTwin 2.0) and real-world (single-arm Franka, dual-arm ARX) environments. Results show that ViDAL consistently improves both regression and flow-matching VLA policies and surpasses competitive action-representation methods.

2 Related Work

Vision-Language-Action Models. Vision-Language-Action (VLA) models adapt pretrained vision-language backbones to predict robot actions from visual observations and language instructions [1–3, 27, 28, 16, 6]. Recent VLAs differ mainly in how their action experts generate actions: flow-matching experts in π_0 [4] and $\pi_{0.5}$ [5] support high-frequency continuous control, while one-step

regression experts in OpenVLA-OFT [23] and StarVLA-OFT [26] target faster inference. ViDAL is orthogonal to these backbones and experts: it changes the action target to a dynamics-grounded latent and decodes the predicted latent back to executable actions, without modifying the underlying VLA architecture.

Action Representations for VLA Models. VLA action representations take three predominant forms, each shaped by a different supervision signal. Raw action chunks are regressed under behavior-cloning [4–7, 29]; discrete action tokens compress trajectories via quantization [8–11]; and continuous action latents use an autoencoder bottleneck without quantization [12]. All three are shaped only by reconstruction, token efficiency, or imitation losses, and rarely consider how the scene visually evolves under each action. ViDAL explicitly introduces visual dynamics as a direct supervision signal for the action latent, anchoring the representation to the visual effect of each action chunk.

Visual Dynamics for VLA Policies. There is also a complementary line that uses visual dynamics to guide VLA policies. JEPA-style methods pretrain on video to provide dynamics-aware features that condition the policy [17, 30]. Future-prediction approaches inject future-frame or video-diffusion signals into the VLM backbone or the policy as auxiliary supervision [18, 31, 32]. Other methods use dynamics or world-model signals for action reasoning [19, 21, 20, 22]. These designs entangle dynamics learning with policy training and tie the visual signal to a VLA architecture, so the dynamics machinery must be retrained for every new backbone. ViDAL confines visual dynamics to the action latent during representation learning and freezes it before policy training, so the same dynamics-grounded Action VAE can be plugged into different VLM backbones and action experts.

3 Method

3.1 Overview

ViDAL learns a reusable action representation whose continuous latent is grounded by visual dynamics. As shown in Figure 2, given a robot trajectory with observation clip $\mathbf{o}_{t:t+H}$ and action chunk $\mathbf{a}_{t:t+H-1}$, an Action VAE encodes the action chunk into a continuous action latent that is jointly supervised by action reconstruction and visual-dynamics prediction based on video VAE model (Section 3.2). Then the frozen Action VAE serves as a plug-in action representation for downstream VLA policy learning (Section 3.3), where the video model is no longer required.

3.2 Learning the Action Latent Space

The central challenge in learning action latents is that action reconstruction alone does not determine which aspects of an action chunk are relevant to policy learning: a low-dimensional latent may reconstruct actions while failing to encode how those actions interact with the current scene. ViDAL addresses this by adding visual-dynamics supervision, encouraging the action latent to be informative for predicting the future visual latent conditioned on the initial observation.

Action VAE. The Action VAE maps the action chunk to a continuous action latent $\mathbf{z}_a \in \mathbb{R}^{T \times D}$ via a Gaussian posterior $q_\phi(\mathbf{z}_a | \mathbf{a}_{t:t+H-1}) = \mathcal{N}(\boldsymbol{\mu}_\phi, \text{diag}(\boldsymbol{\sigma}_\phi^2))$, where the encoder applies a $4 \times$ temporal downsampling so that $T=H/4$ and D is the latent dimension. We sample $\mathbf{z}_a$ with the standard reparameterization trick and decode it back to the original action chunk:

$$\mathbf{z}_a = \boldsymbol{\mu}_\phi + \boldsymbol{\sigma}_\phi \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad \hat{\mathbf{a}}_{t:t+H-1} = D_a^\psi(\mathbf{z}_a). \quad (1)$$

The continuous bottleneck preserves smooth trajectory structure and is compatible with both regression and flow-matching action experts.

Visual-Dynamics Supervision. A frozen video encoder E_v extracts an initial visual latent $\mathbf{z}_0$ from the current observation and a future visual latent $\mathbf{z}_f$ from the subsequent frames as the prediction target,

$$(\mathbf{z}_0, \mathbf{z}_f) = E_v(\mathbf{o}_{t:t+H}), \quad (2)$$

and a lightweight predictor P_θ regresses $\mathbf{z}_f$ from the initial visual latent and the action latent:

$$\hat{\mathbf{z}}_f = P_\theta(\mathbf{z}_0, \mathbf{z}_a). \quad (3)$$

Conditioning on $\mathbf{z}_0$ is essential because the same action chunk can correspond to different scene transitions in different contexts. The predictor therefore drives $\mathbf{z}_a$ to retain action-relevant scene-transition structure rather than only low-level action statistics. Both E_v and P_θ are used only during representation learning and discarded afterwards, so deployment requires no video model.

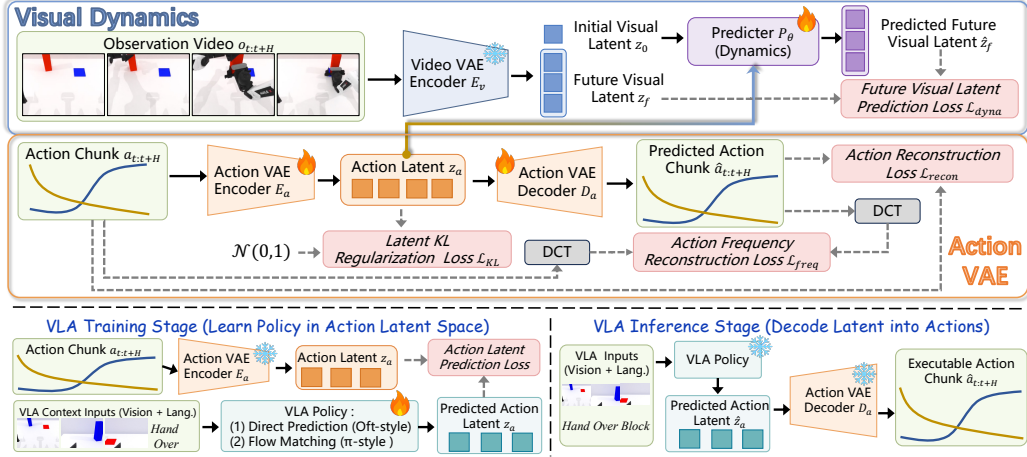


Figure 2: **Framework of ViDAL.** *Top:* the Action VAE is trained with visual-dynamics supervision from a frozen video encoder. *Bottom:* the frozen Action VAE then plugs into VLA training (left) and inference (right), serving as the action representation between the policy and the robot.

Training Objective. The Action VAE is jointly trained with four complementary terms,

$$\mathcal{L}_{\text{ViDAL}} = \mathcal{L}_{\text{rec}} + \lambda_{\text{freq}} \mathcal{L}_{\text{freq}} + \lambda_{\text{KL}} \mathcal{L}_{\text{KL}} + \lambda_{\text{dyna}} \mathcal{L}_{\text{dyna}}, \quad (4)$$

where $\mathcal{L}_{\text{rec}} = \|\hat{\mathbf{a}} - \mathbf{a}\|_1$ reconstructs the action chunk in the time domain, $\mathcal{L}_{\text{freq}} = \|\text{DCT}(\hat{\mathbf{a}}) - \text{DCT}(\mathbf{a})\|_1$ follows FAST’s DCT-based action tokenization [8] to preserve temporal frequency structure, $\mathcal{L}_{\text{KL}} = D_{\text{KL}}(q_\phi(\mathbf{z}_a | \mathbf{a}) \| \mathcal{N}(\mathbf{0}, \mathbf{I}))$ regularizes the latent distribution, and $\mathcal{L}_{\text{dyna}} = \|\hat{\mathbf{z}}_f - \text{sg}(\mathbf{z}_f)\|_1$ aligns the action latent with future scene evolution; the stop-gradient operator $\text{sg}(\cdot)$ keeps the visual target space fixed.

3.3 Integrating ViDAL into VLA Policies

After representation learning, ViDAL replaces the raw-action interface of a VLA policy with an interface based on action latents: during training, the policy predicts an encoded action latent, and at inference, the predicted latent is decoded into an executable action chunk.

Plug-In Action Representation. The Action VAE is frozen and used as a plug-in action representation that the VLA policy is trained to predict. For each action chunk $\mathbf{a}_{t:t+H-1}$, the frozen encoder produces a deterministic latent target,

$$\bar{\mathbf{z}}_a = \text{sg}(\text{Norm}(\mu_\phi(\mathbf{a}_{t:t+H-1}))), \quad (5)$$

where $\text{Norm}(\cdot)$ is a fixed latent normalization estimated from the Action VAE training set, and the bar distinguishes this deterministic downstream target from the sampled latent $\mathbf{z}_a$ used in Action VAE training. This interface keeps the VLM backbone unchanged and only replaces the action-modeling target, so the same dynamics-grounded latent space can be plugged into different VLA architectures without modifying their backbones.

Regression Action Experts. For VLAs with regression action experts such as OpenVLA-OFT [23] and StarVLA-OFT [26], the action expert maps the VLA context $\mathbf{c}$ (visual observation, language instruction, and policy features) to a predicted latent $\tilde{\mathbf{z}}_a$ and is trained against $\bar{\mathbf{z}}_a$:

$$\tilde{\mathbf{z}}_a = F_\omega(\mathbf{c}), \quad \mathcal{L}_{\text{reg}} = \ell(\tilde{\mathbf{z}}_a, \bar{\mathbf{z}}_a), \quad (6)$$

where $\ell(\cdot, \cdot)$ is the regression loss used by the action expert. The lightweight one-step expert is preserved while the supervised target moves from the raw action space to the dynamics-grounded latent space.

Flow-Matching Action Experts. For VLAs with flow-matching action experts such as π_0 and $\pi_{0.5}$ [4, 5], the same latent target serves as the endpoint of a conditional flow. With the interpolant $\bar{\mathbf{z}}_\tau = \tau \bar{\mathbf{z}}_a + (1 - \tau)\xi$ and noise $\xi \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, the action expert learns a vector field in the action latent space:

$$\mathcal{L}_{\text{flow}} = \mathbb{E}_{\tau, \xi, \bar{\mathbf{z}}_a} \left[\left\| v_\omega(\bar{\mathbf{z}}_\tau, \mathbf{c}, \tau) - (\xi - \bar{\mathbf{z}}_a) \right\|_2^2 \right]. \quad (7)$$

The generative process of flow-matching policies is unchanged, and only the modeled distribution shifts from raw actions to dynamics-grounded action latents.

Inference. The VLA policy emits a predicted latent $\tilde{z}_a$ and the frozen Action VAE decoder maps it back to executable actions, $\hat{a}_{t:t+H-1} = D_a^\psi(\tilde{z}_a)$. ViDAL therefore acts as a drop-in action representation layer between any existing VLA action expert and the robot, with no video model in the loop.

4 Experiments

We evaluate ViDAL as a plug-in action representation for VLA policies, demonstrating whether visual-dynamics grounding improves downstream success across simulation and real robots. Following the experimental setup (Section 4.1), the experiments are organized around three questions:

1. Does ViDAL consistently improve VLA policies on simulation benchmarks (Section 4.2), and do the gains transfer to real-robot platforms (Section 4.5)?
2. Which training-objective terms and action-latent design choices drive these improvements (Section 4.3), and how do they scale with tasks and demonstrations (Section 4.4)?
3. Are ViDAL action latents more structured than raw actions, and do they correspond to recognizable future visual dynamics (Section 4.6)?

4.1 Experimental Setup

Benchmarks. We evaluate on two simulation benchmarks and two real-world platforms. LIBERO [24] covers single-arm language-conditioned manipulation across four task suites (Object, Spatial, Goal, Long); RoboTwin 2.0 [25] covers dual-arm manipulation across 50 tasks under a Clean setting matched to training and a Random setting with randomized scene conditions (e.g., background, lighting, distractors, viewpoint). Real-world deployment uses a single-arm Franka and a dual-arm ARX, each with a small set of tabletop tasks; we report task success rates (%) throughout.

VLA backbones and baselines. We pair ViDAL with both regression and flow-matching action experts: OpenVLA-OFT [23] on LIBERO and StarVLA-OFT [26] on RoboTwin 2.0 for regression, and $\pi_{0.5}$ [5] on both benchmarks for flow matching; the primary comparison contrasts raw-action targets against ViDAL latent targets on each backbone. On LIBERO we additionally compare against action-tokenization VLAs (π_0 -FAST [8], FASTER [11]) and recent VLAs that augment the VLM backbone with visual dynamics or world models [33, 19, 34–36].

Training and evaluation. We train ViDAL per benchmark on its demonstrations with a frozen Wan VAE [37] as the video encoder, and reuse it as a frozen plug-in action representation across all VLA backbones. The latent dimension is $D=16$ for the dual-arm settings (RoboTwin 2.0, ARX) and $D=8$ for the single-arm settings (LIBERO, Franka), with loss weights $\lambda_{\text{dyna}}=1$ and $\lambda_{\text{KL}}=10^{-4}$ throughout. Each VLA policy is trained as a multi-task policy across all tasks of its benchmark, following the backbone’s original recipe with the action target replaced by the ViDAL latent.

4.2 Main Results

LIBERO. On LIBERO, ViDAL yields absolute success-rate gains of +0.3% on OpenVLA-OFT and +1.2% on $\pi_{0.5}$, concentrated on the long-horizon Long suite (Table 1). The modest overall margin reflects LIBERO’s near-saturation regime, whereas the Long-suite gain supports our claim that visual-dynamics grounding is most beneficial under extended temporal horizons. $\pi_{0.5}$ +ViDAL still attains the highest average across the comparison, on par with the strongest action-tokenization and recent VLA baselines without action discretization or auxiliary supervision on the VLM backbone.

RoboTwin 2.0. On the dual-arm RoboTwin 2.0, ViDAL yields absolute gains of +11.2%/+9.9% on $\pi_{0.5}$ and +4.2%/+1.8% on StarVLA-OFT under the Clean/Random splits (Table 2). Higher-DoF coordination and longer horizons stress the action representation more than near-saturation LIBERO, and the larger flow-matching gain parallels its distributional action head, which couples more naturally with a continuous dynamics-grounded latent than the one-step regression head of StarVLA-OFT. The gain further persists under Random scene perturbations, allowing the multi-task $\pi_{0.5}$ +ViDAL to surpass the official single-task π_0 specialists on both splits.

Table 1: Success rates (%) on LIBERO. Methods are grouped; gray rows are our +ViDAL variants and deltas are relative to the corresponding backbone.

Method	Object	Spatial	Goal	Long	Average
<i>Action-tokenization VLAs</i>					
π_0 -FAST [8]	97.2	96.6	96.0	86.8	94.2
FASTer [11]	99.4	98.0	98.6	95.4	97.9
<i>Visual-Dynamics Augmented VLAs</i>					
GR00T-N1.5 [33]	97.6	94.4	93.0	90.6	93.9
WorldVLA [19]	96.2	87.6	83.4	60.0	81.8
DreamVLA [34]	94.0	97.5	89.5	89.5	92.6
UniVLA [35]	96.8	96.5	95.6	92.0	95.2
Villa-X [36]	97.0	97.5	91.5	74.5	90.1
<i>VLA backbones (vanilla vs. +ViDAL)</i>					
OpenVLA-OFT [23](1 policy for all 4 suites)	98.0	97.7	96.1	95.3	96.8
OpenVLA-OFT+ViDAL	97.8	98.6	96.2	95.6	97.1(+0.3)
$\pi_{0.5}$ [5]	98.2	98.8	98.0	92.4	96.9
$\pi_{0.5}$ +ViDAL	99.4	99.2	98.2	95.6	98.1(+1.2)

Table 2: Success rates (%) on RoboTwin 2.0 under Clean/Random settings; we show 15 representative tasks and report Average over all 50 tasks. Gray columns are our +ViDAL variants and deltas are relative to the corresponding backbone.

Task	π_0 [4]		StarVLA-OFT [26]		StarVLA-OFT +ViDAL		$\pi_{0.5}$ [5]		$\pi_{0.5}$ +ViDAL	
	Clean	Random	Clean	Random	Clean	Random	Clean	Random	Clean	Random
Adjust Bottle	90.0	56.0	96.0	0.0	100.0	5.0	84.0	73.0	99.0	86.0
Place Empty Cup	37.0	11.0	72.0	4.0	66.0	0.0	80.0	56.0	89.0	83.0
Click Alarmclock	63.0	11.0	91.0	14.0	81.0	6.0	56.0	42.0	79.0	79.0
Open Laptop	85.0	46.0	31.0	0.0	79.0	6.0	75.0	48.0	91.0	77.0
Place Burger Fries	80.0	4.0	96.0	6.0	97.0	4.0	68.0	47.0	85.0	77.0
Dump Bin Bigbin	83.0	24.0	68.0	1.0	87.0	7.0	86.0	54.0	92.0	69.0
Move Playingcard Away	53.0	22.0	69.0	1.0	93.0	5.0	64.0	35.0	73.0	68.0
... (50 tasks total)										
Handover Mic	98.0	13.0	39.0	0.0	90.0	1.0	75.0	26.0	100.0	54.0
Press Stapler	62.0	29.0	60.0	0.0	78.0	11.0	66.0	50.0	83.0	55.0
Open Microwave	80.0	50.0	—	2.0	60.0	15.0	63.0	31.0	75.0	53.0
Stack Bowls Three	66.0	24.0	62.0	0.0	32.0	0.0	53.0	44.0	78.0	52.0
Place Bread Basket	17.0	4.0	52.0	5.0	61.0	2.0	41.0	33.0	57.0	46.0
Pick Dual Bottles	57.0	12.0	43.0	13.0	63.0	8.0	54.0	49.0	69.0	42.0
Put Bottles Dustbin	54.0	13.0	—	0.0	9.0	1.0	47.0	36.0	68.0	42.0
Stack Bowls Two	91.0	41.0	90.0	1.0	67.0	0.0	92.0	66.0	99.0	75.0
Average	46.4	16.3	50.4	2.2	54.6(+4.2)	4.0(+1.8)	54.3	33.2	65.5(+11.2)	43.1(+9.9)

Table 3: Training objective ablation of ViDAL on RoboTwin 2.0 with $\pi_{0.5}$. Clean/Random success rates (%) average over 50 tasks; reconstruction loss is always on.

Variant	$\mathcal{L}_{\text{dyna}}$	$\mathcal{L}_{\text{freq}}$	$\mathcal{L}_{\text{KL}}$	Clean	Random
Vanilla Action VAE			✓	57.4	38.3
w/o $\mathcal{L}_{\text{dyna}}$		✓	✓	62.2	40.1
w/o $\mathcal{L}_{\text{freq}}$	✓		✓	63.1	40.4
w/o $\mathcal{L}_{\text{KL}}$	✓	✓		63.2	39.7
Full ViDAL	✓	✓	✓	65.5	43.1

4.3 Ablation Studies

We ablate ViDAL along two axes: the training objective and the four latent design choices (D , H , λ_{dyna} , λ_{KL}). All ablations retrain the Action VAE from scratch and use $\pi_{0.5}$ as the downstream policy on RoboTwin 2.0; we report Clean/Random average success rates over all 50 tasks.

Training objective. Each loss term in $\mathcal{L}_{\text{ViDAL}}$ complements the others (Table 3). A vanilla Action VAE with only time-domain reconstruction and a KL prior provides a lower bound, and disabling either the DCT-domain reconstruction $\mathcal{L}_{\text{freq}}$ or the visual-dynamics term $\mathcal{L}_{\text{dyna}}$ degrades both splits. The drop is steepest for $\mathcal{L}_{\text{dyna}}$, confirming that visual-dynamics supervision is the structural source of the gain while $\mathcal{L}_{\text{freq}}$ adds a smaller frequency-domain refinement.

Latent design choices. All four design choices show clear trade-offs around the default (Figure 3). The latent dimension D and chunk length H follow inverted-U trends: too small a D or short an H

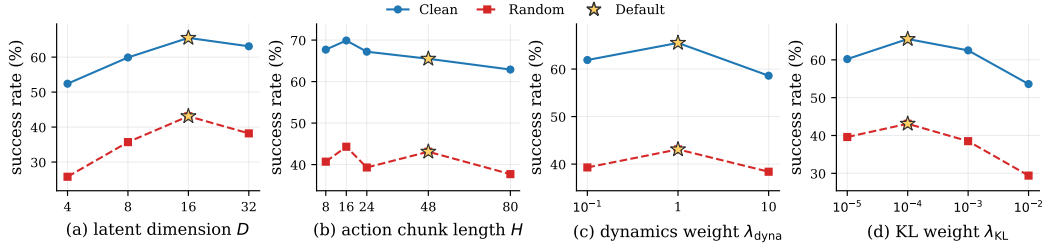


Figure 3: Action latent design ablation of ViDAL on RoboTwin 2.0 with $\pi_{0.5}$. Each panel sweeps one design choice with the others fixed; stars mark the default.

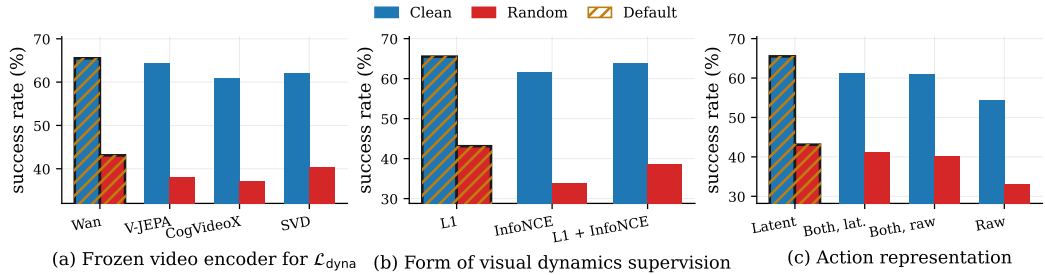


Figure 4: Component comparison for ViDAL on RoboTwin 2.0 with $\pi_{0.5}$. (a) frozen Video Encoder or Video VAE Encoder for $\mathcal{L}_{\text{dyna}}$; (b) form of dynamics supervision; (c) policy prediction target and inference decoding branch (*Latent/Raw*: predict only the latent or only raw actions; *Both, lat./raw*: predict both, decode from the named branch).

underfits, while too large a D or long an H loosens the prior or compresses inconsistent sub-segments into one code. λ_{dyna} behaves similarly: weak supervision loses pixel-space anchoring, while overly strong values divert capacity from action reconstruction. λ_{KL} is the sharpest factor, since an over-strong prior collapses the latent and an over-weak prior leaves it under-regularized for the flow expert. The default thus balances reconstruction, dynamics anchoring, and prior regularization rather than maximizing any single axis.

4.4 Further Analysis

Beyond the latent-design ablations above, we further analyze categorical design choices of ViDAL and how its gain scales with data, under the same controlled $\pi_{0.5}$ /RoboTwin 2.0 protocol as Section 4.3.

Visual-dynamics supervision. The visual-dynamics objective $\mathcal{L}_{\text{dyna}}$ is most effective as a dense L1 regression onto a frozen Wan VAE feature (Fig. 4(a,b)). In panel (a), Wan VAE yields the strongest target among the four frozen video encoders, with CogVideoX [38], SVD [39], and the self-supervised V-JEPA target [17] all narrowing the +ViDAL gain. In panel (b), L1 supervises the action latent with per-feature regression on the entire future video-VAE map, a denser signal than the contrastive InfoNCE objective [40] that only enforces global similarity; replacing L1 with InfoNCE or stacking the two both degrade performance.

Policy prediction target. The frozen ViDAL latent alone is the most effective policy target (Fig. 4(c)). A raw-only head reproduces the $\pi_{0.5}$ baseline [5], while a joint head sits in between regardless of which branch decodes at inference. Splitting head capacity between two targets dilutes the supervision on the dynamics-grounded latent, itself a compact target the flow-matching expert fits on its own.

Scaling with tasks and demonstrations. ViDAL improves $\pi_{0.5}$ at every $|\mathcal{T}|$ and N tested, and the +ViDAL gain widens as data scales up (Fig. 5). Since the eval subset varies with $|\mathcal{T}|$, the comparable trend is read from Δ in panel (b), where the gain widens monotonically; panel (d) shows the same widening of Δ along N . Greater task diversity exposes the action latent to a broader distribution of visual dynamics, while more demonstrations sharpen each anchoring target, so ViDAL is most beneficial in the large-data regime where VLAs are typically trained.

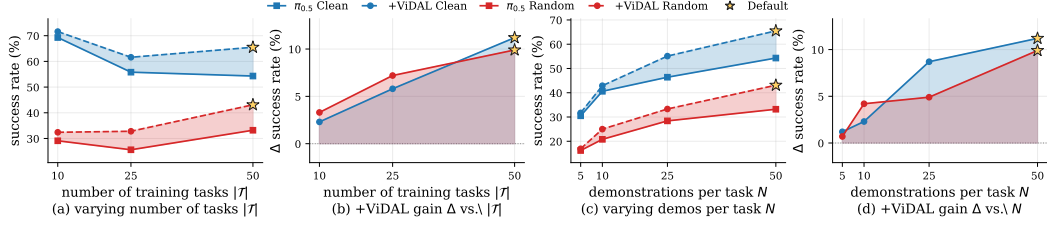


Figure 5: Scaling analysis of ViDAL on RoboTwin 2.0 with $\pi_{0.5}$. (a) success rate against the number of training tasks $|\mathcal{T}|$ (eval subset varies with $|\mathcal{T}|$); (b) the corresponding success rate improvement Δ (+ViDAL- $\pi_{0.5}$), which is comparable across $|\mathcal{T}|$; (c) success rate against demonstrations per task N on a fixed 50-task eval; (d) the corresponding success rate improvement Δ against N .



Figure 6: Real-world deployment of ViDAL on two robot platforms. *Top*: a single-arm Franka Research 3 with a ZED 2i RGB-D camera on four tabletop tasks. *Bottom*: a dual-arm ARX with three RealSense D405 cameras on three bimanual tasks. Each panel shows the hardware setup, representative rollouts, and success rates of $\pi_{0.5}$ with and without ViDAL over 10 rollouts per task.

4.5 Real-Robot Experiments

Setup. We deploy ViDAL on two real-robot platforms to test whether its simulation gains transfer to physical hardware (Fig. 6). The single-arm setup is a Franka Research 3 with a ZED 2i RGB-D camera on four tabletop tasks (*Place Mug on Plate*, *Stack Bowls*, *Place Bowl Close Drawer*, *Place Two Mugs in Basket*), and the dual-arm setup is an ARX with three RealSense D405 cameras on three bimanual tasks (*Fold Towel*, *Wipe Plate*, *Stack Bowls (dual-arm)*). On each platform, the Action VAE is trained on that platform’s own demonstrations and reused as a frozen plug-in action representation, while the underlying $\pi_{0.5}$ policy and its training recipe are kept identical to the simulation runs; the only difference between the two compared variants is whether $\pi_{0.5}$ predicts raw actions or ViDAL latents. Each task is evaluated over 10 rollouts and reported as success rate.

Results. ViDAL consistently improves $\pi_{0.5}$ on every task across both platforms, raising the average success rate by +20.0% on the single-arm Franka and +23.4% on the dual-arm ARX (Fig. 6). The largest single-arm gains appear on tasks that require precise sequencing or insertion under geometric constraints (*Place Bowl Close Drawer*, *Place Two Mugs in Basket*), where the baseline often misjudges approach timing or contact pose; the visually anchored action latent better aligns the predicted action with the resulting scene change, narrowing this failure mode. On the dual-arm side, both deformable manipulation (*Fold Towel*) and rigid bimanual coordination (*Wipe Plate*, *Stack Bowls (dual-arm)*) benefit, and the average gain even exceeds that of the single-arm setup, suggesting that the dynamics-grounded latent is especially useful when contact-rich, higher-DoF behavior must be coordinated across two arms. Together, these results confirm that visual-dynamics grounding transfers cleanly from simulation to real robots, without any retraining or recipe changes to the underlying VLA policy.

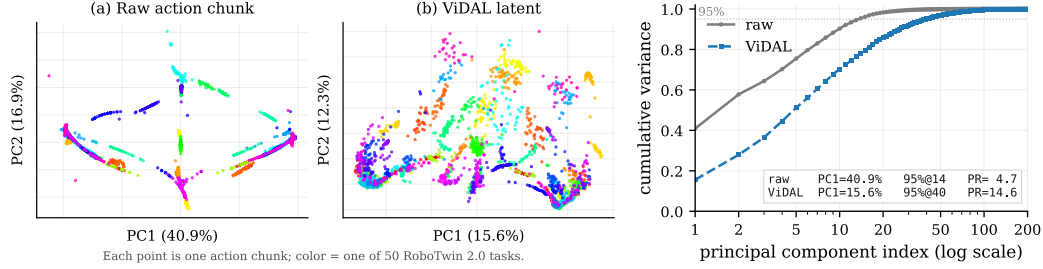


Figure 7: Action latent space geometry on 50 RoboTwin 2.0 tasks. *Left/middle*: 2D PCA projections of raw action chunks and ViDAL latents, colored by task. *Right*: cumulative explained variance vs. principal-component index.

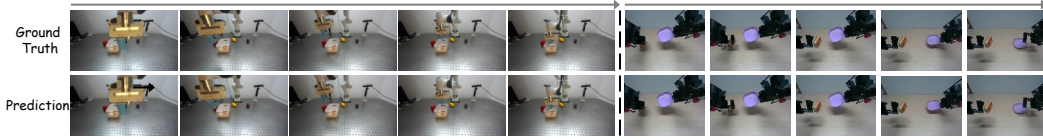


Figure 8: Future video prediction from the ViDAL action latent. For each episode (*left*: real-world Franka; *right*: real-world ARX), top: ground-truth future frames; bottom: frames decoded via the future visual latent predictor and frozen video VAE decoder.

4.6 Qualitative Visualization and Analysis

We examine the learned action latent space qualitatively, asking whether ViDAL yields a richer latent geometry than raw actions and whether it retains enough visual-dynamics information to reconstruct future video frames.

Action latent space geometry. ViDAL produces a less collapsed action latent space than raw actions (Fig. 7). The top principal component absorbs 40.9% of total variance for raw action chunks but only 15.6% for ViDAL, and reaching 95% cumulative variance needs 14 versus 40 components. The participation ratio [41], the effective number of significant latent directions, rises from 4.7 to 14.6 ($3.1\times$). The 2D PCA projections confirm this: raw chunks collapse onto a few low-dimensional curves, whereas ViDAL latents spread across more directions and separate tasks more uniformly. Such a well-regularized latent is the kind of representation that flow-matching action experts learn effectively from [42, 43].

Future video prediction. The same action latent the VLA policy predicts also drives a recognizable future video rollout (Fig. 8). Feeding (z_0, z_a) through the future visual latent predictor P_θ and decoding $\hat{z}_f$ with the frozen video VAE decoder yields frames that track the ground truth in object location, gripper pose, and contact events. The action latent therefore encodes what the scene should look like next, not only what the robot should do. Both the predictor and video decoder are used only for analysis and are not required during policy inference.

5 Conclusion

We propose ViDAL, a method that grounds the action latent space in future visual dynamics by jointly training an Action VAE on action reconstruction and video latent prediction. The frozen Action VAE then serves as a plug-in action representation for both regression and flow-matching VLA policies without modifying the VLM backbone, and consistently improves performance across two simulation benchmarks (LIBERO, RoboTwin 2.0) and two real-robot platforms (single-arm Franka, dual-arm ARX), and further analysis confirms that the advantage widens as tasks and demonstrations scale up. Since ViDAL decouples action-representation learning from VLA policy training, extending it to diverse action spaces and embodiments requires only retraining the lightweight Action VAE; leveraging large-scale egocentric human videos as an additional source of visual-dynamics supervision is a natural next step toward a universal dynamics-grounded action latent space.

References

- [1] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.
- [2] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.
- [3] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [4] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [5] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, James Tanner, Quan Vuong, Homer Walke, Anna Walling, Haohuan Wang, Lili Yu, and Ury Zhilinsky. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [6] Songming Liu, Lingxuan Wu, Bangguo Li, Hengkai Tan, Huayu Chen, Zhengyi Wang, Ke Xu, Hang Su, and Jun Zhu. Rdt-1b: a diffusion foundation model for bimanual manipulation. *arXiv preprint arXiv:2410.07864*, 2024.
- [7] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.
- [8] Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. Fast: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025.
- [9] Yating Wang, Haoyi Zhu, Mingyu Liu, Jiange Yang, Hao-Shu Fang, and Tong He. Vq-vla: Improving vision-language-action models via scaling vector-quantized action tokenizers. pages 11089–11099, 2025.
- [10] Hongyi Zhou, Weiran Liao, Xi Huang, Yucheng Tang, Fabian Otto, Xiaogang Jia, Xinkai Jiang, Simon Hilber, Ge Li, Qian Wang, et al. Beast: Efficient tokenization of b-splines encoded action sequences for imitation learning. *arXiv preprint arXiv:2506.06072*, 2025.
- [11] Yicheng Liu, Shiduo Zhang, Zibin Dong, Baijun Ye, Tianyuan Yuan, Xiaopeng Yu, Linqi Yin, Chenhao Lu, Junhao Shi, Luca Jiang-Tao Yu, et al. Faster: Toward efficient autoregressive vision language action modeling via neural action tokenization. *arXiv preprint arXiv:2512.04952*, 2025.
- [12] Wenhui Tan, Bei Liu, Junbo Zhang, Ruihua Song, and Jianlong Fu. Rold: Robot latent diffusion for multi-task policy modeling. pages 340–353, 2024.
- [13] Homer Rich Walke, Kevin Black, Tony Z Zhao, Quan Vuong, Chongyi Zheng, Philippe Hansen-Estruch, Andre Wang He, Vivek Myers, Moo Jin Kim, Max Du, et al. Bridgedata v2: A dataset for robot learning at scale. In *Conference on Robot Learning*, pages 1723–1736. PMLR, 2023.
- [14] Abby O’Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, Ajinkya Jain, et al. Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0. pages 6892–6903, 2024.
- [15] Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, Mohan Kumar Srirama, Lawrence Yunliang Chen, Kirsty Ellis, et al. Droid: A large-scale in-the-wild robot manipulation dataset. *arXiv preprint arXiv:2403.12945*, 2024.
- [16] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, et al. Octo: An open-source generalist robot policy. 2024.

- [17] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zhohlov, et al. V-jepa 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.
- [18] Yucheng Hu, Yanjiang Guo, Pengchao Wang, Xiaoyu Chen, Yen-Jen Wang, Jianke Zhang, Koushil Sreenath, Chaochao Lu, and Jianyu Chen. Video prediction policy: A generalist robot policy with predictive visual representations. 2024.
- [19] Jun Cen, Chaohui Yu, Hangjie Yuan, Yuming Jiang, Siteng Huang, Jiayan Guo, Xin Li, Yibing Song, Hao Luo, Fan Wang, et al. Worldvla: Towards autoregressive action world model. *arXiv preprint arXiv:2506.21539*, 2025.
- [20] Xiaoxu Xu, Hao Li, Jinhui Ye, Yilun Chen, Jia Zeng, Xinyi Chen, Linning Xu, Dahua Lin, Weixin Li, and Jiangmiao Pang. Futurevla: Joint visuomotor prediction for vision-language-action model. *arXiv preprint arXiv:2603.10712*, 2026.
- [21] Shuyao Shang, Bing Zhan, Yunfei Yan, Yuqi Wang, Yingyan Li, Yasong An, Xiaoman Wang, Jierui Liu, Lu Hou, Lue Fan, et al. Dynvla: Learning world dynamics for action reasoning in autonomous driving. *arXiv preprint arXiv:2603.11041*, 2026.
- [22] Yue Su, Sijin Chen, Haixin Shi, Mingyu Liu, Zhengshen Zhang, Ningyuan Huang, Weiheng Zhong, Zhengbang Zhu, Yuxiao Liu, and Xihui Liu. World guidance: World modeling in condition space for action generation. *arXiv preprint arXiv:2602.22010*, 2026.
- [23] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.
- [24] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.
- [25] Tianxing Chen, Zhanxin Chen, Baijun Chen, Zijian Cai, Yibin Liu, Zixuan Li, Qiwei Liang, Xianliang Lin, Yiheng Ge, Zhenyu Gu, et al. Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. 2025.
- [26] Jinhui Ye, Ning Gao, Senqiao Yang, Jinliang Zheng, Zixuan Wang, Yuxin Chen, Pengguang Chen, Yilun Chen, Shu Liu, and Jiaya Jia. StarVLA- α : Reducing complexity in vision-language-action systems. *arXiv preprint arXiv:2604.11757*, 2026.
- [27] Danny Driess, Fei Xia, Mehdi SM Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, et al. Palm-e: An embodied multimodal language model. 2023.
- [28] Xinghang Li, Minghuan Liu, Hanbo Zhang, Cunjun Yu, Jie Xu, Hongtao Wu, Chilam Cheang, Ya Jing, Weinan Zhang, Huaping Liu, et al. Vision-language foundation models as effective robot imitators. 2023.
- [29] Tony Z Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.
- [30] Jingwen Sun, Wenya Zhang, Zekun Qi, Shaojie Ren, Zezhi Liu, Hanxin Zhu, Guangzhong Sun, Xin Jin, and Zhibo Chen. Vla-jepa: Enhancing vision-language-action model with latent world model. *arXiv preprint arXiv:2602.10098*, 2026.
- [31] Yueru Jia, Jiaming Liu, Shengbang Liu, Rui Zhou, Wanhe Yu, Yuyang Yan, Xiaowei Chi, Yandong Guo, Boxin Shi, and Shanghang Zhang. Video2act: A dual-system video diffusion policy with robotic spatio-motional modeling. *arXiv preprint arXiv:2512.03044*, 2025.
- [32] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, You Liang Tan, Chuning Zhu, Jiannan Xiang, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026.
- [33] Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- [34] Wenya Zhang, Hongsi Liu, Zekun Qi, Yunnan Wang, Xinqiang Yu, Jiazha Zhang, Runpei Dong, Jiawei He, Fan Lu, He Wang, et al. Dreamvla: a vision-language-action model dreamed with comprehensive world knowledge. *arXiv preprint arXiv:2507.04447*, 2025.

- [35] Qingwen Bu, Yanting Yang, Jisong Cai, Shenyuan Gao, Guanghui Ren, Maoqing Yao, Ping Luo, and Hongyang Li. Univla: Learning to act anywhere with task-centric latent actions. *arXiv preprint arXiv:2505.06111*, 2025.
- [36] Xiaoyu Chen, Hangxing Wei, Pushi Zhang, Chuheng Zhang, Kaixin Wang, Yanjiang Guo, Rushuai Yang, Yucen Wang, Xinquan Xiao, Li Zhao, et al. Villa-x: enhancing latent action modeling in vision-language-action models. *arXiv preprint arXiv:2507.23682*, 2025.
- [37] Team Wan, Ang Wang, Baole Ai, Bin Wen, Chaojie Mao, Chen-Wei Xie, Di Chen, Feiwei Yu, Haiming Zhao, Jianxiao Yang, Jianyuan Zeng, Jiayu Wang, Jingfeng Zhang, Jingren Zhou, Jinkai Wang, Jixuan Chen, Kai Zhu, Kang Zhao, Keyu Yan, Lianghua Huang, Mengyang Feng, Ningyi Zhang, Pandeng Li, Pingyu Wu, Ruihang Chu, Ruili Feng, Shiwei Zhang, Siyang Sun, Tao Fang, Tianxing Wang, Tianyi Gui, Tingyu Weng, Tong Shen, Wei Lin, Wei Wang, Wei Wang, Wenmeng Zhou, Wenten Wang, Wenting Shen, Wenyuan Yu, Xianzhong Shi, Xiaoming Huang, Xin Xu, Yan Kou, Yangyu Lv, Yifei Li, Yijing Liu, Yiming Wang, Yingya Zhang, Yitong Huang, Yong Li, You Wu, Yu Liu, Yulin Pan, Yun Zheng, Yuntao Hong, Yupeng Shi, Yutong Feng, Zeyinzi Jiang, Zhen Han, Zhi-Fan Wu, and Ziyu Liu. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025.
- [38] Zhuoyi Yang, Jiayan Teng, Wendi Zheng, Ming Ding, Shiyu Huang, Jiazheng Xu, Yuanming Yang, Wenyi Hong, Xiaohan Zhang, Guanyu Feng, et al. Cogvideox: Text-to-video diffusion models with an expert transformer. *arXiv preprint arXiv:2408.06072*, 2024.
- [39] Andreas Blattmann, Tim Dockhorn, Sumith Kulal, Daniel Mendelevitch, Maciej Kilian, Dominik Lorenz, Yam Levi, Zion English, Vikram Voleti, Adam Letts, et al. Stable video diffusion: Scaling latent video diffusion models to large datasets. *arXiv preprint arXiv:2311.15127*, 2023.
- [40] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018.
- [41] Peiran Gao and Surya Ganguli. On simplicity and complexity in the brave new world of large-scale neuroscience. *Current opinion in neurobiology*, 32:148–155, 2015.
- [42] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022.
- [43] Tero Karras, Miika Aittala, Jaakko Lehtinen, Janne Hellsten, Timo Aila, and Samuli Laine. Analyzing and improving the training dynamics of diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 24174–24184, 2024.

A Method details

Action VAE architecture. The Action VAE maps a normalized action chunk $\mathbf{a}_{t:t+H-1} \in \mathbb{R}^{H \times d_a}$ to a short sequence of continuous action-latent tokens. Its encoder is a temporal convolutional network with residual blocks and strided temporal downsampling, so the latent sequence preserves the temporal order of the action chunk while reducing its length. The encoder predicts a Gaussian posterior with mean and variance parameters, from which the action latent is sampled during representation learning. For downstream VLA supervision, we use the posterior mean as a deterministic action-latent target. The decoder mirrors the encoder with temporal upsampling and reconstructs the original action chunk from the latent sequence. This design keeps the action representation continuous, temporally structured, and directly decodable into executable robot actions.

Frozen video-latent target. For each action chunk, we use the aligned observation clip $\mathbf{o}_{t:t+H}$, which contains one more frame than the action horizon. A frozen video VAE encodes this clip into spatiotemporal latent tokens. For a clip with $H+1 = 4k+1$ frames, image size $h \times w$, and video-latent channel dimension C_v , the flattened video latent has shape

$$\mathbf{z}_v \in \mathbb{R}^{T_v N_s \times C_v}, \quad T_v = k + 1, \quad N_s = (h/16)(w/16). \quad (8)$$

We split this latent into the first-frame spatial tokens $\mathbf{z}_0 \in \mathbb{R}^{N_s \times C_v}$ and the future target $\mathbf{z}_f \in \mathbb{R}^{(T_v-1)N_s \times C_v}$. The first-frame tokens provide the current visual context, while the future tokens provide the visual-dynamics target induced by the action chunk.

Future-latent predictor. The dynamics predictor learns to infer the future visual latent from the current visual context and the action latent. It represents the current and future visual positions as a shared token sequence, conditions the future queries on the action-latent tokens through cross-attention, and predicts the future visual-latent tokens jointly:

$$\hat{\mathbf{z}}_f = P_\theta(\mathbf{z}_0, \mathbf{z}_a). \quad (9)$$

Conditioning on $\mathbf{z}_0$ disambiguates actions whose visual effects depend on the current scene, while conditioning on $\mathbf{z}_a$ encourages the action latent to encode the scene transition caused by the action chunk. The predictor is used only during representation learning and is not part of the deployed VLA policy.

Training objective. The Action VAE is trained with four terms:

$$\mathcal{L}_{\text{ViDAL}} = \mathcal{L}_{\text{rec}} + \lambda_{\text{freq}} \mathcal{L}_{\text{freq}} + \lambda_{\text{KL}} \mathcal{L}_{\text{KL}} + \lambda_{\text{dyna}} \mathcal{L}_{\text{dyna}}. \quad (10)$$

The reconstruction term is an L1 loss on normalized actions, $\mathcal{L}_{\text{rec}} = \|\hat{\mathbf{a}} - \mathbf{a}\|_1$. The frequency-domain term applies a discrete cosine transform along the temporal axis and computes an L1 distance between reconstructed and target frequency coefficients. The KL term is the standard Gaussian-posterior regularizer. The visual-dynamics term is an L1 loss on frozen video-latent tokens,

$$\mathcal{L}_{\text{dyna}} = \|P_\theta(\mathbf{z}_0, \mathbf{z}_a) - \text{sg}(\mathbf{z}_f)\|_1, \quad (11)$$

where $\text{sg}(\cdot)$ denotes that the video-latent target is held fixed. The weights and schedules are benchmark-specific and are listed in Section B.

Downstream VLA interface. After representation learning, the video VAE and future-latent predictor are discarded. The frozen Action VAE defines the interface between a VLA policy and the robot: its encoder converts each action chunk into a deterministic latent target, and its decoder converts predicted latents back into executable action chunks. We normalize the action latents before policy training and invert the normalization before decoding at inference. Thus, downstream VLA training changes only the action target space; the VLM backbone and the policy action expert remain otherwise unchanged.

B Implementation details

Action VAE training. We train a separate Action VAE for each benchmark or robot platform, using the same architecture and objective described in Section A. Table 4 summarizes the main configuration choices. Single-arm settings use the LIBERO configuration, and the Franka real-robot setting follows the same configuration. Dual-arm settings use the RoboTwin 2.0 configuration, and

Table 4: Key Action VAE training configurations. We report only the settings that define the action-latent interface and visual-dynamics target.

Setting	LIBERO / Franka	RoboTwin 2.0 / ARX
Action dimension d_a	7	14
Action chunk length H	32	48
Action latent dimension D	8	16
Observation clip length	33 frames	49 frames
Image resolution	128×128	96×128
Video latent tokens $T_v \times N_s$	9×64	13×48
Training steps	50k	50k

the ARX real-robot setting follows the same configuration. Across settings, we use a frozen Wan video VAE as the visual-dynamics target, $\lambda_{\text{dyna}}=1$, $\lambda_{\text{freq}}=1$, and $\lambda_{\text{KL}}=10^{-4}$.

Downstream VLA integration. For all downstream policies, ViDAL only changes the action representation: the raw action chunk target is replaced by the normalized Action VAE latent, and the predicted latent is decoded by the frozen Action VAE decoder before execution. The original VLA training and inference recipes are otherwise kept unchanged for OpenVLA-OFT and StarVLA-OFT. For $\pi_{0.5}$, we keep the original flow-matching objective and inference procedure, but set the action expert learning rate to $10 \times$ the VLM learning rate. This adjustment helps the pretrained action expert adapt from the raw-action distribution used in $\pi_{0.5}$ pretraining to the ViDAL latent distribution. At inference, neither the video VAE nor the future-latent predictor is used; the only additional deployed module is the Action VAE decoder, whose latency overhead is negligible compared with the VLA forward pass.

Training data and evaluation. On LIBERO, we use the official training demonstrations and official evaluation protocol. On RoboTwin 2.0, we train on the Clean demonstrations from 50 tasks, with 50 episodes per task and 2,500 episodes in total; evaluation follows the official Clean and Random settings, with 100 rollouts per task in each setting. For real-robot experiments, the single-arm Franka setting contains four tasks with 50 demonstrations per task, and the dual-arm ARX setting contains three tasks with 100 demonstrations per task. Each real-robot task is evaluated with 10 rollouts, and a rollout is counted as successful if the robot completes the language instruction for that task.

C Full RoboTwin 2.0 results

Table 5 reports the full per-task RoboTwin 2.0 results corresponding to Table 2. All methods are evaluated under both Clean and Random settings, with 100 rollouts per task in each setting. The representative rows shown in the main paper are a subset of this full table, and the averages match Table 2 up to rounding.

D Future prediction examples

Figures 9 and 10 show additional future visual prediction examples from the learned action latent across simulation and real-robot data. For each example, the top row shows ground-truth future frames from the trajectory, and the bottom row shows frames decoded from the predicted future video latent conditioned on the ViDAL action latent. These visualizations are used only as qualitative analysis of the latent space; the video VAE, future-latent predictor, and video decoder are not part of the deployed policy.

Table 5: Full RoboTwin 2.0 success rates (%) over all 50 tasks. Gray columns indicate the corresponding +ViDAL variants.

Task	π_0		StarVLA-OFT		StarVLA-OFT +ViDAL		$\pi_{0.5}$		$\pi_{0.5}$ +ViDAL	
	Clean	Rand.	Clean	Rand.	Clean	Rand.	Clean	Rand.	Clean	Rand.
Adjust Bottle	90.0	56.0	96.0	0.0	100.0	5.0	84.0	73.0	99.0	86.0
Beat Block Hammer	43.0	21.0	58.0	1.0	68.0	2.0	68.0	19.0	86.0	28.0
Blocks Ranking RGB	19.0	5.0	45.0	0.0	9.0	0.0	48.0	22.0	71.0	41.0
Blocks Ranking Size	7.0	1.0	27.0	0.0	3.0	0.0	26.0	7.0	37.0	11.0
Click Alarmclock	63.0	11.0	91.0	14.0	81.0	6.0	56.0	42.0	79.0	79.0
Click Bell	44.0	3.0	94.0	4.0	98.0	16.0	64.0	77.0	53.0	51.0
Dump Bin Bigbin	83.0	24.0	68.0	1.0	87.0	7.0	86.0	54.0	92.0	69.0
Grab Roller	96.0	80.0	93.0	14.0	92.0	20.0	96.0	87.0	100.0	89.0
Handover Block	45.0	8.0	0.0	0.0	27.0	0.0	32.0	1.0	49.0	13.0
Handover Mic	98.0	13.0	39.0	0.0	90.0	1.0	75.0	26.0	100.0	54.0
Hanging Mug	11.0	3.0	15.0	0.0	4.0	0.0	9.0	8.0	16.0	11.0
Lift Pot	84.0	36.0	0.0	0.0	56.0	0.0	58.0	2.0	89.0	4.0
Move Can Pot	58.0	21.0	50.0	0.0	71.0	2.0	30.0	11.0	62.0	28.0
Move Pillbottle Pad	21.0	1.0	54.0	0.0	72.0	0.0	35.0	17.0	47.0	23.0
Move Playingcard Away	53.0	22.0	69.0	1.0	93.0	5.0	64.0	35.0	73.0	68.0
Move Stapler Pad	0.0	2.0	12.0	0.0	5.0	0.0	23.0	12.0	12.0	13.0
Open Laptop	85.0	46.0	31.0	0.0	79.0	6.0	75.0	48.0	91.0	77.0
Open Microwave	80.0	50.0	–	2.0	60.0	15.0	63.0	31.0	75.0	53.0
Pick Diverse Bottles	27.0	6.0	30.0	7.0	42.0	8.0	51.0	40.0	63.0	30.0
Pick Dual Bottles	57.0	12.0	43.0	13.0	63.0	8.0	54.0	49.0	69.0	42.0
Place A2B Left	31.0	1.0	20.0	0.0	62.0	2.0	66.0	26.0	62.0	39.0
Place A2B Right	27.0	6.0	22.0	2.0	44.0	3.0	52.0	17.0	59.0	35.0
Place Bread Basket	17.0	4.0	52.0	5.0	61.0	2.0	41.0	33.0	57.0	46.0
Place Bread Skillet	23.0	1.0	56.0	2.0	52.0	2.0	36.0	21.0	52.0	29.0
Place Burger Fries	80.0	4.0	96.0	6.0	97.0	4.0	68.0	47.0	85.0	77.0
Place Can Basket	41.0	5.0	63.0	0.0	68.0	1.0	52.0	3.0	62.0	12.0
Place Cans Plasticbox	34.0	2.0	81.0	2.0	87.0	3.0	28.0	32.0	19.0	33.0
Place Container Plate	88.0	45.0	99.0	3.0	94.0	2.0	95.0	50.0	94.0	76.0
Place Dual Shoes	15.0	0.0	28.0	1.0	2.0	0.0	33.0	10.0	23.0	7.0
Place Empty Cup	37.0	11.0	72.0	4.0	66.0	0.0	80.0	56.0	89.0	83.0
Place Fan	20.0	10.0	28.0	0.0	20.0	0.0	57.0	24.0	44.0	25.0
Place Mouse Pad	7.0	1.0	9.0	0.0	14.0	0.0	24.0	23.0	31.0	25.0
Place Object Basket	16.0	2.0	40.0	1.0	46.0	0.0	65.0	15.0	72.0	19.0
Place Object Scale	10.0	0.0	19.0	0.0	32.0	1.0	54.0	28.0	60.0	38.0
Place Object Stand	36.0	11.0	48.0	2.0	83.0	0.0	77.0	31.0	80.0	55.0
Place Phone Stand	35.0	7.0	24.0	0.0	63.0	1.0	34.0	35.0	58.0	39.0
Place Shoe	28.0	6.0	63.0	0.0	14.0	1.0	58.0	45.0	64.0	42.0
Press Stapler	62.0	29.0	60.0	0.0	78.0	11.0	66.0	50.0	83.0	55.0
Put Bottles Dustbin	54.0	13.0	–	0.0	9.0	1.0	47.0	36.0	68.0	42.0
Put Object Cabinet	68.0	18.0	35.0	0.0	34.0	0.0	25.0	31.0	41.0	28.0
Rotate QRCode	68.0	15.0	50.0	0.0	45.0	0.0	64.0	23.0	71.0	23.0
Scan Object	18.0	1.0	13.0	1.0	30.0	1.0	16.0	14.0	32.0	18.0
Shake Bottle	99.0	51.0	98.0	11.0	100.0	23.0	95.0	87.0	100.0	98.0
Shake Bottle Horizontally	97.0	60.0	98.0	10.0	100.0	36.0	96.0	94.0	99.0	99.0
Stack Blocks Three	17.0	0.0	41.0	0.0	13.0	0.0	37.0	0.0	59.0	26.0
Stack Blocks Two	42.0	1.0	83.0	0.0	67.0	1.0	59.0	15.0	83.0	43.0
Stack Bowls Three	66.0	24.0	62.0	0.0	32.0	0.0	53.0	44.0	78.0	52.0
Stack Bowls Two	91.0	41.0	90.0	1.0	67.0	0.0	92.0	66.0	99.0	75.0
Stamp Seal	3.0	4.0	27.0	0.0	25.0	0.0	25.0	11.0	51.0	31.0
Turn Switch	27.0	23.0	26.0	0.0	25.0	6.0	24.0	31.0	39.0	15.0
Average	46.4	16.3	50.4	2.2	54.6	4.0	54.3	33.2	65.5	43.1

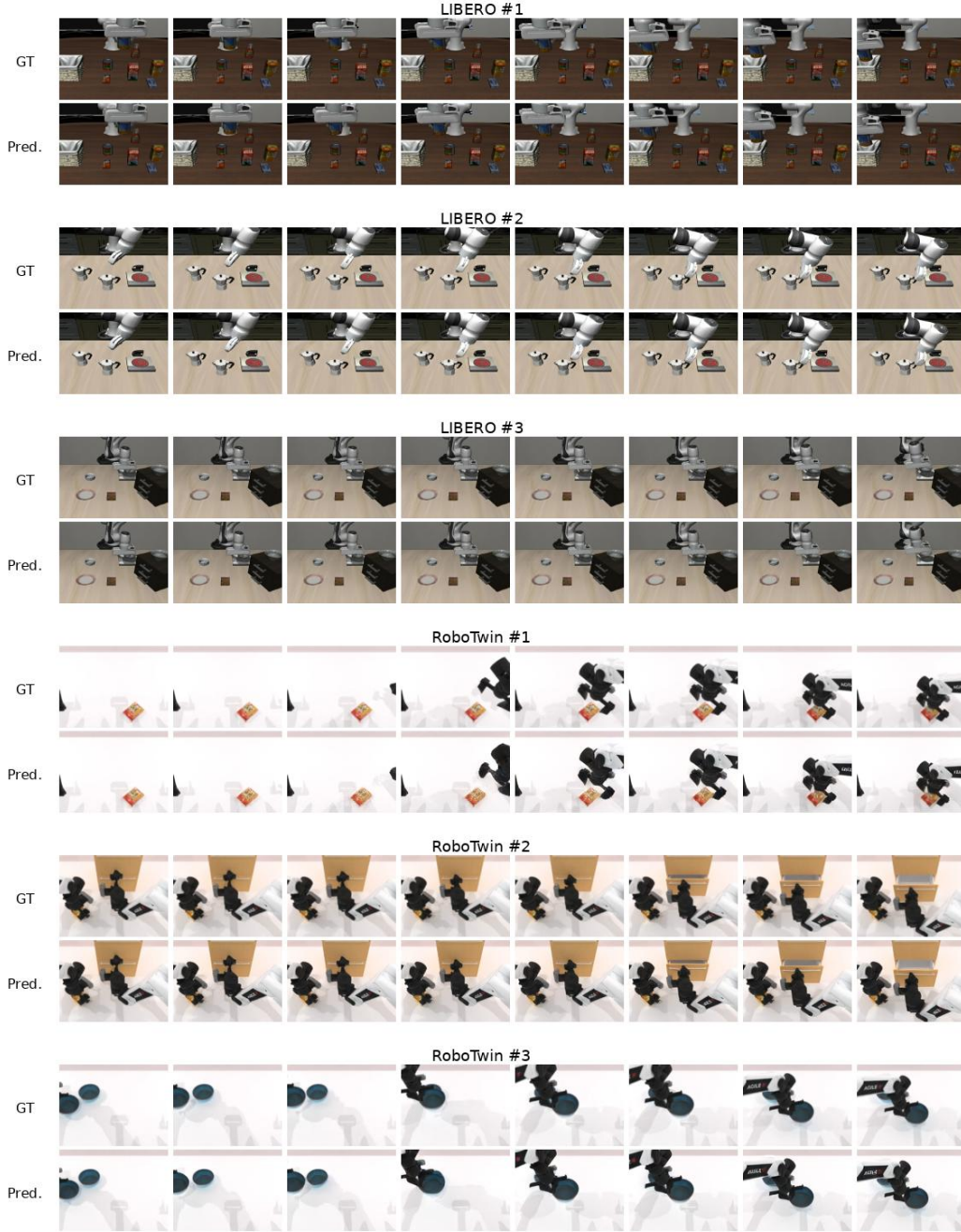


Figure 9: Future visual prediction examples on simulation benchmarks (LIBERO and RoboTwin 2.0). Each example contains a ground-truth row and a prediction row decoded from the future visual latent predicted using the ViDAL action latent.

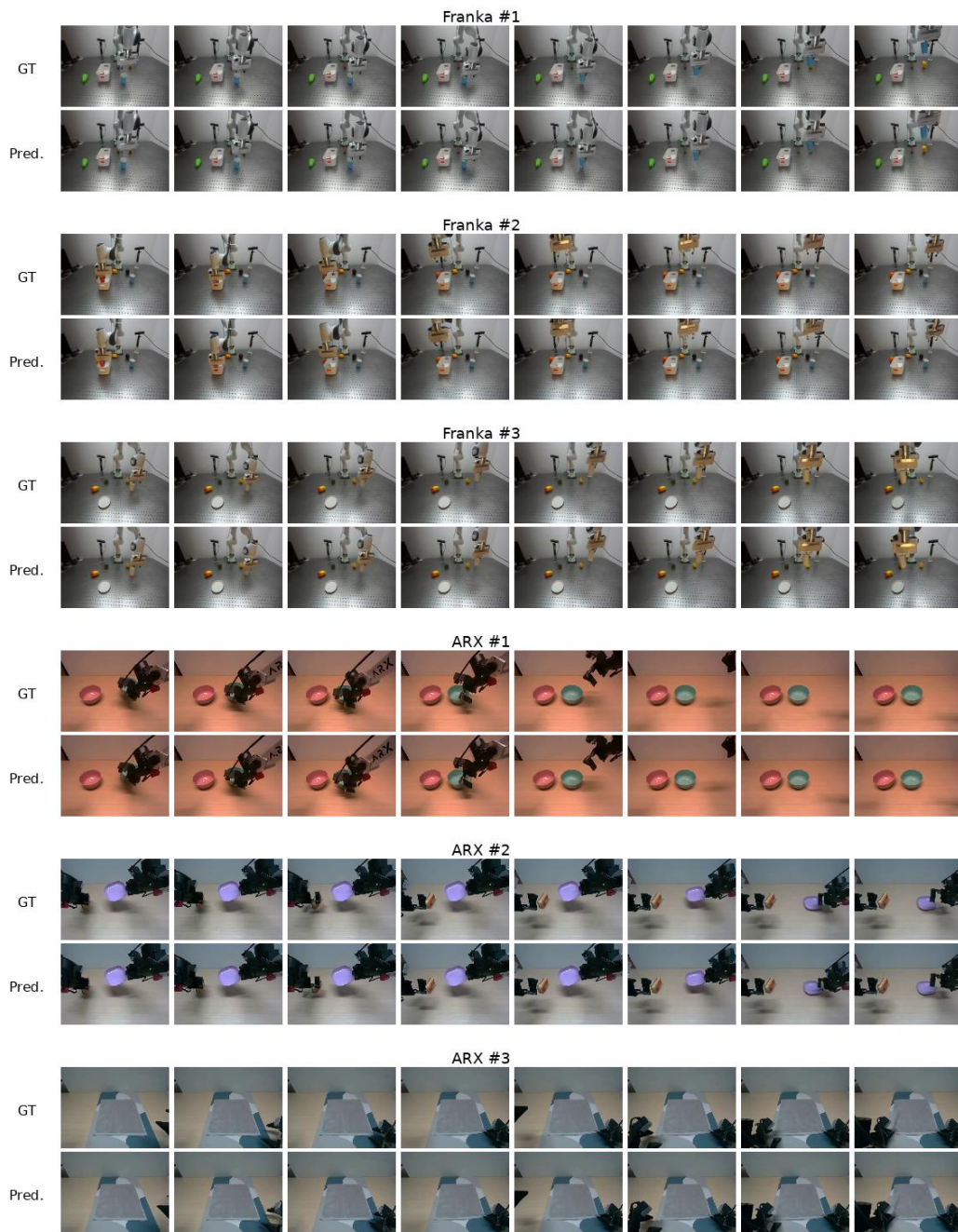


Figure 10: Future visual prediction examples on real-robot datasets (Franka and ARX). Each example contains a ground-truth row and a prediction row decoded from the future visual latent predicted using the ViDAL action latent.